



Systematic Literature Review for the
project “The impact of service-mesh
solutions on the efficiency of
microservice applications”

Alla Krylova 196722

Oleksandr Nychyporchuk 196659

Kiryl Pashkevich 196687

Pavel Khmialeuski 197055

Wprowadzenie

Ewolucja świata: od monolitu do mikroservisów

Zalety architektury mikroservisowej

- Skalowalność i Niezawodność: Podstawa nowoczesnych systemów rozproszonych.
- Time-to-market: Szybsze wdrożenia dzięki niezależności zespołów.

Wezwania i trudności

- Nieprzewidywalność komunikacji w chmurze.
- Złożona konfiguracja i trudne wdrażanie polityk bezpieczeństwa.
- Problemy z pozyskiwaniem spójnej telemetrii (monitoringu).

Service Mesh to dedykowana warstwa infrastruktury, która oddziela logikę biznesową aplikacji od zarządzania siecią. Rozwiązania tego typu posiadają kilka cech:

- Architektura Sidecar Proxy
 - Mikrouслуги nie komunikują się bezpośrednio.
 - Cały ruch przechodzi przez lokalne obiekty pomocnicze (Proxy).
- Automatyzacja i Odporność:
 - Routing i Discovery: Automatyczne wykrywanie usług i kierowanie ruchu.
 - Niezawodność: Wbudowane mechanizmy Circuit Breaker oraz Timeouts.

- Bezpieczeństwo i Monitoring:
 - Szyfrowanie ruchu (mTLS) oraz negocjowanie połączeń.
 - Spójne raportowanie telemetrii do centralnej warstwy zarządzania.

Popularne implementacje Service Mesh

- Istio - rozbudowane i popularne rozwiązanie. Oferuje potężny zestaw funkcji: od zaawansowanego routingu po szczegółową analitykę (telemetrię).
- Linkerdv2 - skoncentrowane na prostocie i wysokiej wydajności. Dzięki proxy napisanym w języku Rust, minimalizuje narzut na zasoby systemowe i opóźnienia.
- Consul - rozwiązanie przeważające w środowiskach hybrydowych. W przeciwieństwie do rozwiązań czysto kubernetesowych, płynnie łączy mikrouslugi z maszynami wirtualnymi i serwerami bare metal.

Opis poszczególnych etapów przebiegu

Krok 1: Cele i Pytania (1)

- Jakie rozwiązania Service Mesh są najczęściej stosowane w projektach rzeczywistych?
- Które metryki (opóźnienia, przepustowość, wykorzystanie zasobów) są najczęściej oceniane podczas porównywania rozwiązań Service Mesh i klasycznych?
- Jakie mechanizmy bezpieczeństwa (szyfrowanie, autoryzacja, polityki) te rozwiązania zapewniają i jak duży jest spadek wydajności w porównaniu do natywnego zarządzania bezpieczeństwem Kubernetes?
- Jakie wzorce deploy-owania i strategii migracji są proponowane w przypadku wprowadzenia Service Mesh w ramach już istniejącego rozwiązania

Krok 1: Cele i Pytania (2)

- Czym się różni monitorowanie i zbieranie telemetrii pomiędzy rozwiązaniami wykorzystującymi i niewykorzystującymi Service Mesh

Krok 2: Słowa kluczowe do wyszukiwania

- Podstawowa technologia: “Service mesh”, “Service meshes”
- Przestrzeń porównywania: “without service mesh”, “standard Kubernetes”, “baseline”, “vs. native”
- Konkretnie implementacje: Istio, Linderd, Consul
- Odniesienia do technologii: “microservices architecture”, “micro-service*”, “micro services”, “Kubernetes”
- Aspekty wydajnościowe: performance, latency, throughput, overhead, scalability, efficiency

Krok 3: Wybrane bazy wiedzy

- Scopus
- SpringerLink
- Arxiv
- IEEE Xplore

Krok 4: Kryteria włączenia

- Typ publikacji: artykuł naukowy recenzowany, publikacje konferencyjne
- Język: angielski
- Okres lat: 2017-2026 (aktywny rozwój rozwiązań Service Mesh zaczął się w 2017 wraz z udostępnieniem Istio)
- Dostępność: pełny tekst

Krok 5: Kryteria wykluczenia

- Długość publikacji: mniejsza od 4 stron
- Istotność: publikacje, w których Service Mesh jest tematem drugorzędnym albo jego parametry nie są badane

- Przejrzystość opisu: czy architektura, postępowanie eksperymentu czy metodologia zostały opisane w dokładny i zrozumiały sposób
- Ocena ilościowa: czy publikacja dostarcza pomiary ilościowe wydajności (metryki procesora, opóźnienia, pamięci, przepustowości)
- Porównanie z rozwiązaniem no service mesh: czy w publikacji rozwiązania service mesh są porównywane z no service mesh albo pomiędzy sobą
- Rzetelność eksperymentu: czy warunki eksperymentu (obciążenie pracą, środowisko, wymagania sprzętowe software-owe) zostały opisane w dokładny sposób

Krok 6: Kryteria oceny jakości (2)

- Walidacje statystyczne: istotność statystyczna (Statistical significance), przedziały ufności (confidence intervals), powtarzalność (reproducibility)
- Analiza bezpieczeństwa: czy publikacja zawiera analizę bezpieczeństwa (security analysis) albo ocenę podatności (vulnerability assessment)
- Źródło publikacji: czy artykuł został opublikowany w renomowanym źródle

Krok 7: Dane do ekstrakcji (1)

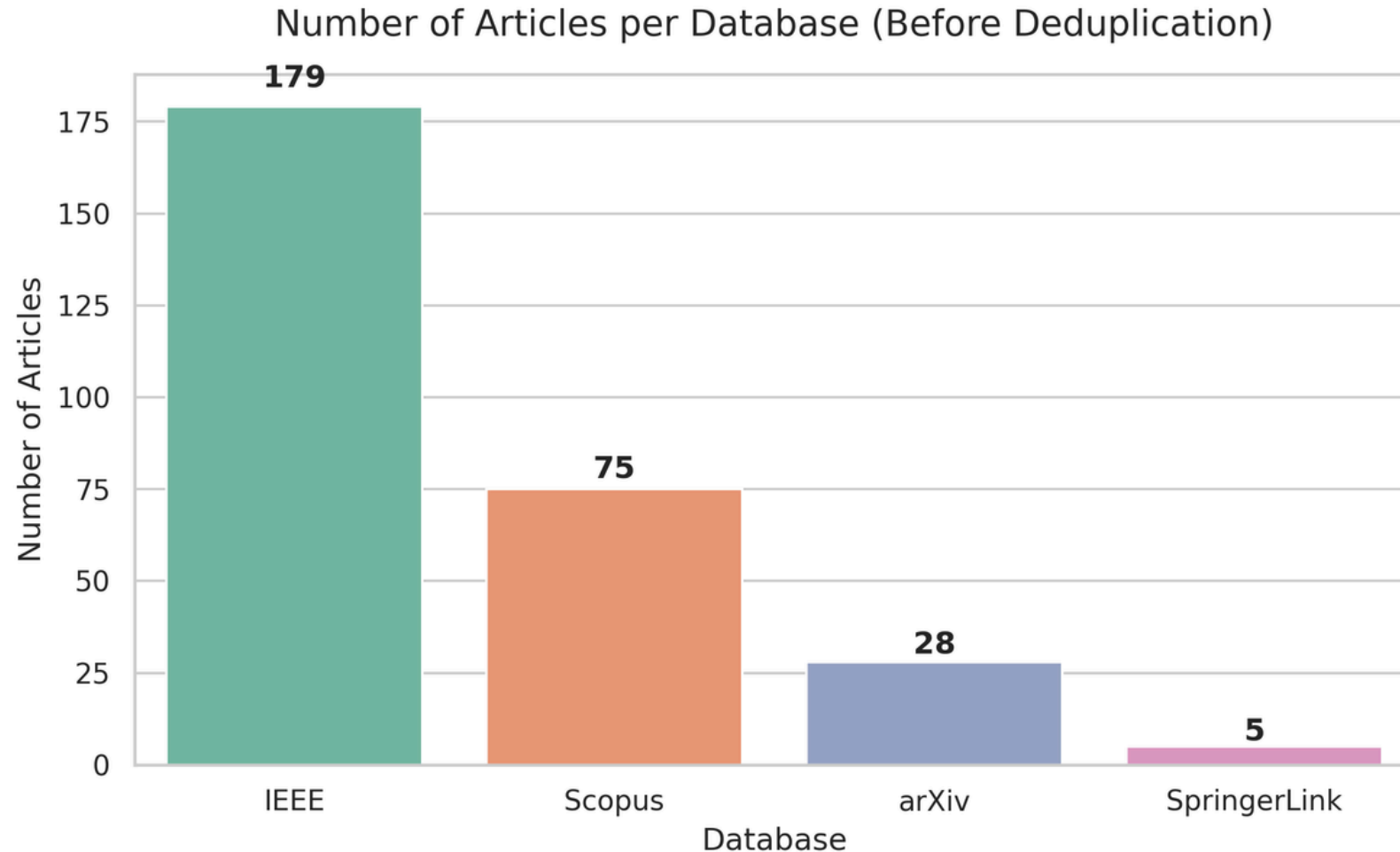
- Metadane publikacji (id publikacji, DOI, tytuł, rok wydania, autorzy)
- Kontekst badania (rozwiązania SM i bazowe, platforma orkestracji, typ aplikacja do testowania)
- Metodologia (metoda badawcza, obciążenie systemu)
- Metryki wydajności (opóźnienia, przepustowość, wykorzystanie procesora i pamięci, narzut danych)

Krok 7: Dane do ekstrakcji (2)

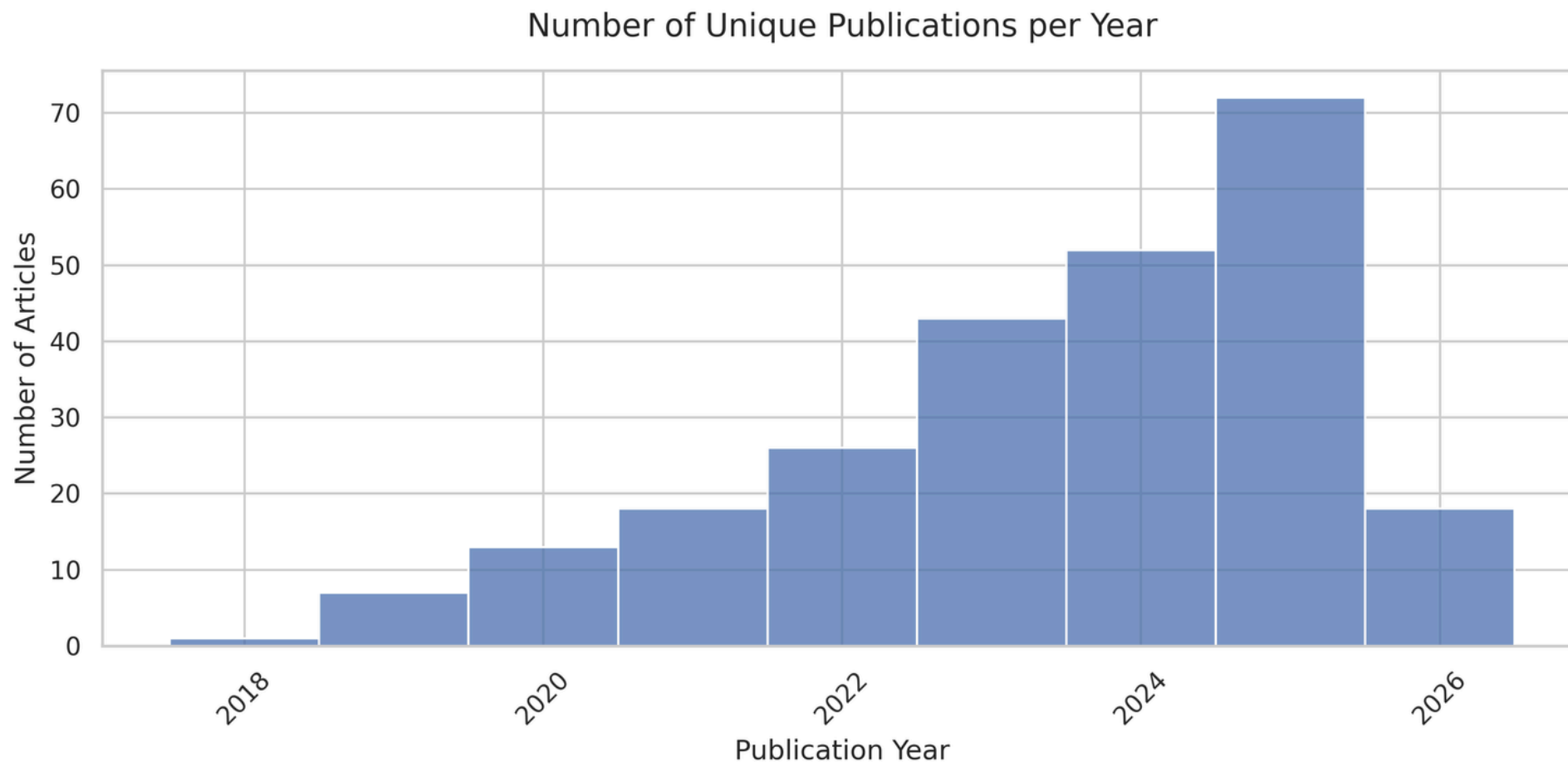
- Aspekty bezpieczeństwa (mechanizmy bezpieczeństwa, wpływ na wydajność)
- Obserwowalność systemu (narzędzia do monitorowania, składniki obserwowalności)
- Wyniki (wnioski, ograniczenia, wskazany kierunki w przyszłych pracach)

Statystyki

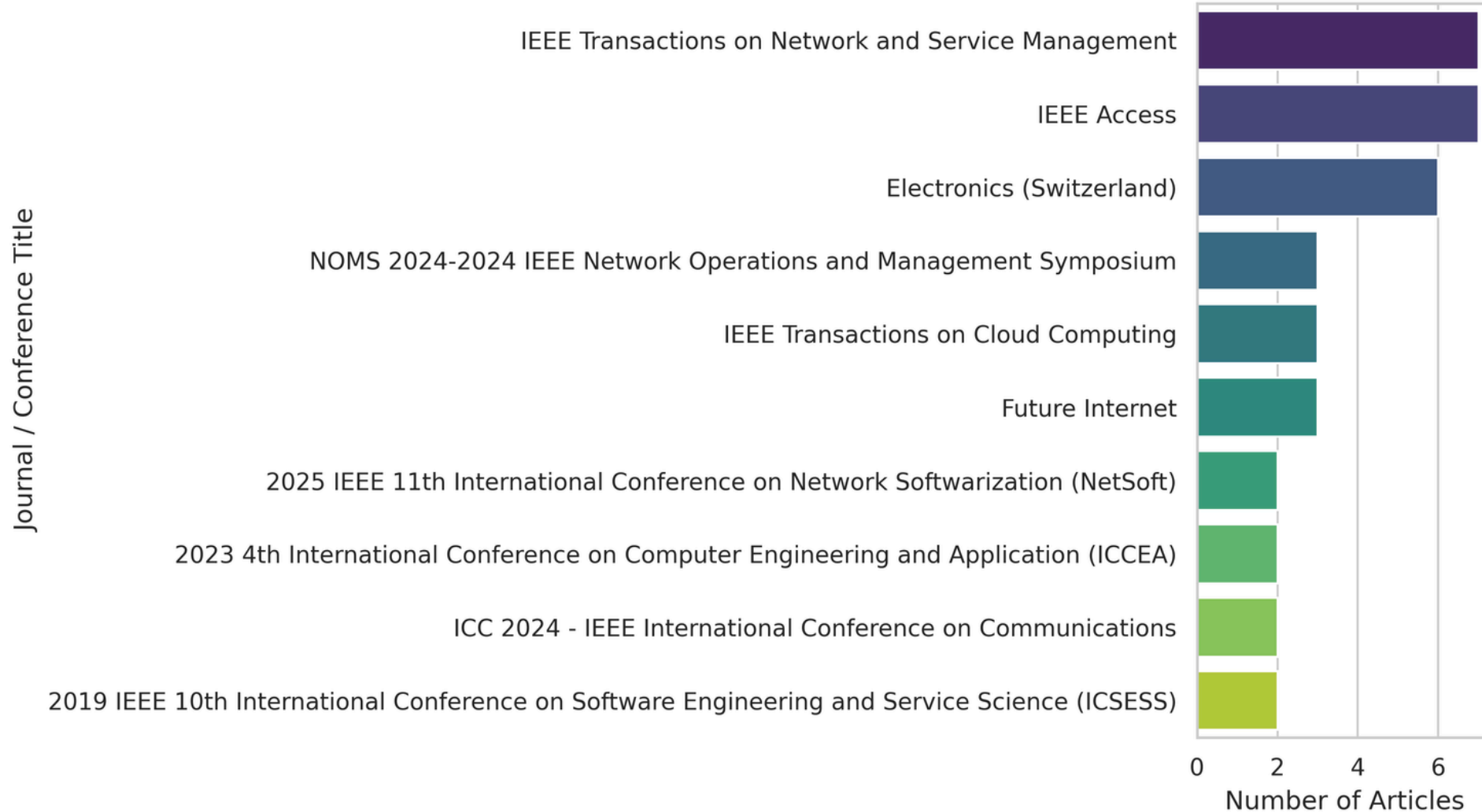
Rozkład wyników w różnych bazach



Trendy czasowe



Top 10 Publication Sources (Unique Articles)



Analiza wybranych publikacji

- Tytuł: Edge computing, performance benchmarking, latency, throughput, Iptables overhead
- Zidentyfikowany problem: Istio/Envoy może zwiększyć opóźnienia nawet 3-krotnie, co ogranicza jego użycie w systemach czasu rzeczywistego.
- Wykryte wąskie gardło: Zidentyfikowano niską wydajność algorytmów iptables w jądrze Linux przy dużej skali ruchu.
- Wniosek: Przyszłość Service Mesh na krawędzi sieci (Edge) wymaga akceleracji sprzętowej (Smart-NIC) oraz optymalizacji ścieżki danych w kernelu.

- Tytuł: Resilient microservices: an investigation into Istio effectiveness in Kubernetes
 - Kryteria wydajność: Linkerd cechuje się niższymi opóźnieniami i pełną stabilnością przy 300 współbieżnych użytkownikach.
 - Zużęte zasoby: Istio wykazuje znacznie wyższe zużycie RAM i piki CPU, co w testach prowadziło do awarii serwera.
 - Guideline-y do wyboru: Linkerd to rozwiązanie optymalne dla środowisk o ograniczonych zasobach. Istio pozostaje opcją dla projektów wymagających najbardziej zaawansowanej kontroli.

- Tytuł: Technical Report: Performance Comparison of Service Mesh Frameworks: the mTLS Test Case
 - Szyfrowania mTLS: Udowodniono, że sam protokół mTLS ma znikomy wpływ na latencję (<3%).
 - Koszt architektury: Tradycyjny model Sidecar generuje duże opóźnienia przez dodatkowe skoki sieciowe (network hops).
 - Przełom: Podejście Sidecarless (eBPF) redukuje narzut latencji do zaledwie 8%, łącząc wysoką przepustowość ze standardami Zero-Trust.

- Tytuł: Comparative Evaluation of Linkerd and Istio Service Meshes in a Microservices Architecture Application
- Główne źródło spadku wydajności: To nie szyfrowanie, lecz analiza HTTP (L7 Parsing) i zbieranie telemetrii najbardziej obciążają procesor.
- Spróby optymalizacja: Wyłączenie inspekcji L7 w Istio pozwoliło na 5-krotny wzrost przepustowości systemu.
- Skalowalność systemu: Zużycie pamięci rośnie liniowo wraz z liczbą połączeń. Kluczem do sukcesu jest balans między głęboką analityką a responsywnością systemu.



Dziękujemy za uwagę

